Numerical method for solving the one dimensional wave equation

Leonardo Toffalini

1 Problem statement

The one dimensional wave equation on the interval $[0, \frac{\pi}{2}]$ looks as follows

$$\begin{cases} \partial_{tt}u(t,x) = \partial_{xx}u(t,x) + f & (x \in (0, \frac{\pi}{2}), \quad t \in (0,1)) \\ \partial_t u(0,x) = 0 & (x \in (0, \frac{\pi}{2})) \\ u(0,x) = \cos x & (x \in (0, \frac{\pi}{2})) \\ \partial_x u(t,0) = 0 & (t \in (0,1)) \\ u(t, \frac{\pi}{2}) = t^2 & (t \in (0,1)) \end{cases}$$

where $f \equiv 2$.

The analytic solution is $u(t,x) = t^2 + \cos t \cos x$.

1.1 Numerical method

We are going to rely on the second order approximation of ∂_{xx} and ∂_{tt} which look as follows

$$\partial_{xx}u(t,x) = \frac{1}{h_x^2}(u(t, x+h_x) - 2u(t,x) + u(t, x-h_x)) + O(h_x^2)$$

$$\partial_{tt}u(t,x) = \frac{1}{\delta^2}(u(t+\delta, x) - 2u(t,x) + u(t-\delta, x)) + O(\delta^2),$$

where we discretize in space with h_x and δ in time.

With the above finite difference schemes we can rewrite the exact equation into a numerical procedure

$$\frac{1}{\delta^2}(u(t+\delta, x) - 2u(t,x) + u(t-\delta, x)) = \frac{1}{h_x^2}(u(t, x+h_x) - 2u(t,x) + u(t, x-h_x)) + f.$$

Rearranging the above equation to have only $u(t+\delta, x)$ on the left hand side we get

$$u(t+\delta, x) = 2u(t,x) - u(t-\delta, x) + \frac{\delta^2}{h_x^2}(u(t, x+h_x) - 2u(t,x) + u(t, x-h_x)) + \delta^2 f.$$

Let us define $t_n := n\delta$ and $x_k := kh_x$, with these $u_k^n := u(t_n, x_k)$. Using these notations we can simplify to

$$u_k^{n+1} = 2u_k^n - u_k^{n-1} + \frac{\delta^2}{h_x^2}(u_{k+1}^n - 2u_k^n + u_{k-1}^n) + \delta^2 f. \tag{1}$$

Note, that the error terms collect to form $\delta^2(O(\delta^2) + O(h_x^2))$ in total.

1.2 Handling the boundary conditions

The easiest boundary condition to handle is $u(0,x) = \cos x$, which just means that we initialize $u_k^0 = \cos kh_x$.

Handling the boundary condition $u(t, \frac{\pi}{2}) = t^2$ is just as trivial, we just set $u^n_{N_x-1} = t_n^2$ in each time step, where N_x is the number of grid points in the space discretization.

Next, we handle $\partial_t u(0, x) = 0$. Notice, that in Equation 1 to get u_k^{t+1} we need u_k^n and u_k^{n-1}, which means that we need two initial values to start our numerical method: u_k^0 and u_k^1. To combat this, let us write out the second order Taylor approximation of $u(t, x)$ around $t = 0$:

$$u(\delta, x) = u(0, x) + \delta \overbrace{\partial_t u(0, x)}^{0} + \delta^2 \partial_{tt} u(0, x) + O(\delta^3).$$

We can see that the first order term vanishes due to the Neumann boundary condition, and by the problem statement we know that

$$\partial_{tt} u(0, x) = \partial_{xx} u(0, x) + f,$$

where we already have an approximation for $\partial_{xx} u(0, x)$. Thus we conclude that to initialize u_k^1 we can use the following formula

$$u_k^1 = u_k^0 + \frac{\delta^2}{h_x^2}(u_{k+1}^0 - 2u_k^0 + u_{k-1}^0) + \delta^2 f.$$

The only boundary condition left to handle is $\partial_x u(t, 0) = 0$. Notice that for u_0^{n+1} we would need u_{-1}^n alongside u_1^n and u_0^n. We can instead use the Neumann boundary condition $\partial_x u(t, 0) = 0$ to solve this problem.

Let us write out the central finite difference of ∂_x to get some equation for u_{-1}^n

$$\partial_x u_0^n = \frac{1}{2h_x}(u_1^n - u_{-1}^n) + O(h_x^2).$$

Since we know that $\partial_x u_0^n = 0$ we have that $u_1^n = u_{-1}^n + O(h_x^3)$ for all time steps. With this trick we can resolve the previous problem by giving a modified formula for the left end of the space interval

$$\begin{aligned}
\partial_{xx} u_0^{n+1} &= \frac{1}{h_x^2}(u_1^n - 2u_0^n + u_{-1}^n) + O(h_x^2) \\
&= \frac{1}{h_x^2}(u_1^n - 2u_0^n + u_1^n + O(h_x^3)) + O(h_x^2) \\
&= \frac{2}{h_x^2}(u_1^n - u_0^n) + O(h_x) + O(h_x^2).
\end{aligned}$$

We see that when handling the Neumann boundary condition in space we introduce a first order error at the boundary, which naturally proposes the question whether this error propagates inward or stays localized at the boundary. Luckily, the global consistency error stays the same, which can be checked numerically, see Figure 1.

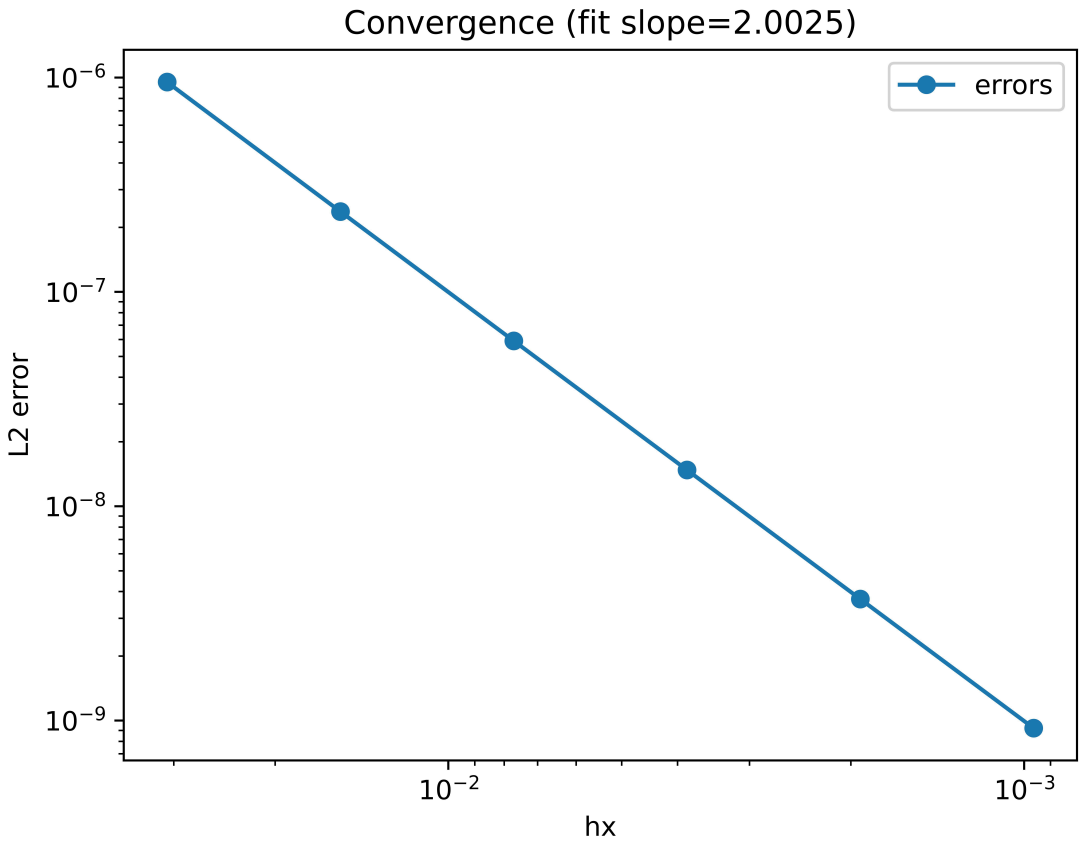
In conclusion, we have handled the left and right ends of the space interval, the left side was handled by $\partial_x u(t, 0) = 0$, while the right side was handled by $u(t, \frac{\pi}{2}) = t^2$.

We have also handled the start of the time frame, where we needed two initial values, one of which was provided as is $u(0, x) = \cos x$, and the second we figured out how to compute from the Neumann condition $\partial_t u(0, x) = 0$.

1.3 Numerical experiments

In our numerical experiments we implemented the above method and estimated the convergence rate of the method by refining the subdivisions and comparing against the analytic solution.

It is important to note, that while refining h_x we made sure to adjust δ such that δ/h_x stays the same. We aimed to keep this ratio just under 1.0, since this ensures stability.

The results can be seen in the following table, or more visually in Figure 1.

Level	h_x	δ	$\frac{\delta^2}{h_x^2}$	L^2-error	rate
0	$3.0799 \cdot 10^{-2}$	$3.0303 \cdot 10^{-2}$	0.9679	$9.5270 \cdot 10^{-7}$	
1	$1.5399 \cdot 10^{-2}$	$1.5151 \cdot 10^{-2}$	0.9679	$2.3701 \cdot 10^{-7}$	2.0071
2	$7.6999 \cdot 10^{-3}$	$7.5757 \cdot 10^{-3}$	0.9679	$5.9108 \cdot 10^{-8}$	2.0035
3	$3.8499 \cdot 10^{-3}$	$3.7878 \cdot 10^{-3}$	0.9679	$1.4758 \cdot 10^{-8}$	2.0018
4	$1.9249 \cdot 10^{-3}$	$1.8939 \cdot 10^{-3}$	0.9679	$3.6874 \cdot 10^{-9}$	2.0009
5	$9.6249 \cdot 10^{-4}$	$9.4696 \cdot 10^{-4}$	0.9679	$9.2159 \cdot 10^{-10}$	2.0004